UNIVERSIDAD CATÓLICA DE TEMUCO
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA INFORMÁTICA

INFORME DEL PROYECTO

Diseño e Implementación de un Analizador Sintáctico LL(1)

Tema: SmartRule - DSL para Automatización IoT y Domótica

Dato	**Información**
Asignatura	Teoría de la Computación
Código curso	INFO1148
Integrantes	José Sáez (Jefe de grupo) Catalina Vergara Alexa Galeano
Profesor	Prof. M. Lévano
Fecha de entrega	01/11/2026

Temuco, Chile

Índice

1. Introducción **5**
1.1. Antecedentes . . . 5
1.2. Planteamiento del Problema . . . 5
1.3. Objetivo General . . . 5
1.4. Objetivos Específicos . . . 5

2. Dominio del Lenguaje: SmartRule **7**
2.1. Descripción General . . . 7
2.2. Justificación del Dominio . . . 7
2.3. Alcance del Lenguaje . . . 7
2.4. Vocabulario Léxico: Palabras Reservadas y Tokens . . . 7
2.5. Alfabeto del Lenguaje . . . 8
2.6. Ejemplos de Programas . . . 8
2.6.1. Ejemplos Válidos . . . 8
2.6.2. Ejemplos Inválidos (Detectados por el Analizador) . . . 8
2.7. Extensión Creativa . . . 9

3. Gramática Libre de Contexto (GLC) **10**
3.1. Definición Formal . . . 10
3.2. Producciones de la Gramática . . . 10
3.3. Análisis y Transformaciones de la Gramática . . . 10
3.3.1. Recursión Izquierda . . . 10
3.3.2. Factorización por la Izquierda . . . 11
3.3.3. Ambigüedad y Precedencia . . . 11

4. Conjuntos FIRST, FOLLOW y Tabla LL(1) **12**
4.1. Conjuntos FIRST . . . 12
4.2. Conjuntos FOLLOW . . . 12
4.3. Tabla LL(1) y Verificación de Conflictos . . . 13

5. Diseño e Implementación **16**
5.1. Arquitectura del Proyecto . . . 16
5.2. Analizador Léxico . . . 16
5.3. Analizador Sintáctico LL(1) Descendente Recursivo . . . 17
5.4. Árbol Sintáctico Abstracto (AST) . . . 17
5.5. Manejo de Errores . . . 17

6. Pruebas, Ejecución y Resultados **18**
6.1. Ejecución del Analizador . . . 18
6.1.1. Modo Interactivo . . . 18
6.1.2. Analizar archivo con salida completa . . . 18
6.1.3. Verificar FIRST, FOLLOW y Tabla LL(1) . . . 18
6.2. Suite de Pruebas Automatizadas . . . 18
6.3. Casos de Prueba Documentados . . . 19
6.4. Resultados de las Pruebas . . . 21
6.5. Ejemplo de Salida del Analizador . . . 21

7. Conclusiones **23**

8. Recomendaciones y Trabajo Futuro **23**

9. Referencias Bibliográficas **24**

10. Anexos **25**
10.1. Anexo A: Repositorio del Proyecto 25
10.2. Anexo B: Instrucciones para Compilar y Ejecutar 25
10.2.1. Requisitos 25
10.2.2. Ejecución 25
10.3. Anexo C: Gramática Completa en Formato BNF 26
10.4. Anexo D: Derivación por la Izquierda 27
10.4.1. Objetivo 27
10.4.2. Gramática utilizada 27
10.4.3. Derivación por la izquierda 27
10.4.4. Cadena derivada 28
10.5. Anexo E: Demostración Funcional 30

Índice de tablas

1. Tokens definidos para el lenguaje SmartRule 7
2. Integración de la extensión creativa en gramática, tabla LL(1) y pruebas . 9
3. Conjuntos FIRST calculados 12
4. Conjuntos FOLLOW calculados 13
5. Producciones numeradas y conjuntos de selección 13
6. Tabla predictiva LL(1): cada celda contiene el número de la producción de la Tabla 5; las celdas vacías indican error sintáctico 15
7. Nodos del AST implementados 17
8. Distribución de pruebas automatizadas 19
9. Casos de prueba documentados (10 válidos, 10 inválidos y 5 límite) 19

Índice de figuras

1. Estructura del proyecto con arquitectura limpia 16
2. Ejecución de pruebas en el terminal integrado de VS Code 30
3. Ejecución del analizador en el terminal integrado de VS Code 31
4. Ejecución del analizador: continuación del AST 32
5. Detección de errores sintácticos en el terminal integrado de VS Code . . . 33
6. Producciones numeradas de la gramática 34
7. Conjuntos FIRST, FOLLOW y tabla LL(1) (59 entradas, 0 conflictos) . . . 35
8. Detección de un error léxico en el terminal integrado de VS Code 36

1. Introducción

Detrás de todo compilador hay un análisis sintáctico que revisa que el código respete una estructura definida, y en Teoría de la Computación vimos que esa estructura se describe con gramáticas libres de contexto. Este proyecto fue la oportunidad de llevarlo a la práctica.

Creamos SmartRule, un lenguaje para escribir reglas de automatización del hogar inteligente, con una lógica simple de "cuando pase esto, entonces haz esto otro", más un "si no" opcional. Para este lenguaje diseñamos e implementamos un analizador sintáctico LL(1) descendente recursivo, con su gramática, conjuntos FIRST y FOLLOW, tabla predictiva, árbol sintáctico abstracto (AST) y mensajes de error claros, todo validado con 68 pruebas automatizadas.

1.1. Antecedentes

Todo compilador o intérprete se apoya en la teoría de autómatas y los lenguajes formales [1]. Dentro de ese proceso, el análisis sintáctico (o *parsing*) cumple un rol clave: revisar que la secuencia de tokens que entrega el analizador léxico respete la estructura gramatical del lenguaje.

Para describir esa estructura de manera formal se usan las Gramáticas Libres de Contexto (GLC), que sirven tanto para los lenguajes de programación como para los Lenguajes de Dominio Específico (*Domain Specific Languages - DSL*), que están pensados para resolver problemas de un área en particular.

1.2. Planteamiento del Problema

Necesitábamos construir un analizador sintáctico para un lenguaje de dominio específico creado por nosotros. Este debía reconocer y validar los programas escritos en ese lenguaje, avisar cuando hubiera errores sintácticos con mensajes fáciles de entender, y representar la estructura de lo que se analiza mediante un Árbol Sintáctico Abstracto (AST).

1.3. Objetivo General

Diseñar e implementar un analizador sintáctico LL(1) descendente recursivo para el lenguaje de dominio específico *SmartRule*. Para lograrlo, aplicamos Gramáticas Libres de Contexto, conjuntos *FIRST* y *FOLLOW* y análisis LL(1), además de construir el Árbol Sintáctico Abstracto (AST) y detectar los errores sintácticos del programa.

1.4. Objetivos Específicos

- Definir el dominio, propósito, tokens, palabras reservadas y sintaxis del lenguaje *SmartRule*.
- Diseñar una Gramática Libre de Contexto (GLC) correcta, no ambigua, completa y coherente con el lenguaje propuesto.
- Analizar la gramática, identificando y corrigiendo recursividad, factorización y posibles ambigüedades.

- Calcular y justificar los conjuntos *FIRST* y *FOLLOW* para cada no-terminal.
- Construir la tabla predictiva LL(1), verificando la ausencia de conflictos.
- Implementar un analizador léxico (extendido a partir del cuaderno base entregado por el docente) y un analizador sintáctico LL(1) descendente recursivo desarrollado por el grupo.
- Construir un Árbol Sintáctico Abstracto (AST) que represente correctamente la estructura del programa.
- Implementar un mecanismo de detección y reporte de errores léxicos y sintácticos con mensajes claros y localizados.
- Desarrollar un conjunto de pruebas exhaustivo ($>$25 casos), incluyendo casos válidos, inválidos, casos límite y una extensión creativa integrada a la gramática.
- Documentar todo el proceso mediante un informe técnico en LaTeX y un `README.md` reproducible.

2. Dominio del Lenguaje: SmartRule

2.1. Descripción General

SmartRule es un Lenguaje de Dominio Específico (DSL) pensado para escribir reglas de automatización en sistemas de domótica e Internet de las Cosas (IoT). Su sintaxis sigue la lógica de la lengua natural: *"SI ocurre una condición, ENTONCES ejecutar una o más acciones"*. Además, incluye una cláusula opcional *SINO* (`ELSE`) para indicar qué hacer cuando la condición no se cumple.

2.2. Justificación del Dominio

Elegimos el hogar inteligente porque es un tema conocido y fácil de entender. Con este dominio pudimos mostrar de forma clara los conceptos vistos en el curso: reconocimiento de patrones, gramáticas, análisis predictivo LL(1), árboles sintácticos y manejo de errores. Además, es lo bastante simple como para mantener una gramática LL(1) válida, pero con la flexibilidad suficiente para agregar una extensión creativa.

2.3. Alcance del Lenguaje

El lenguaje *SmartRule* permite:

- Definir múltiples reglas secuenciales en un mismo archivo.
- Evaluar condiciones compuestas mediante operadores lógicos `AND`, `OR` y `NOT`.
- Realizar comparaciones relacionales entre variables, sensores o valores numéricos.
- Ejecutar una o más acciones separadas por comas.
- Realizar asignaciones a actuadores o variables (`variable = valor`).
- Invocar acciones con o sin argumentos.
- Definir un bloque alternativo mediante la palabra reservada `ELSE`.
- Trabajar con valores booleanos `TRUE` y `FALSE`.

2.4. Vocabulario Léxico: Palabras Reservadas y Tokens

Categoría	**Elementos**
Palabras Reservadas	`WHEN`, `THEN`, `ELSE`, `AND`, `OR`, `NOT`, `TRUE`, `FALSE`
Identificadores	`[A-Za-z_][A-Za-z0-9_]*`
Números	`[0-9]+`
Operadores Relacionales	`>`, `<`, `>=`, `<=`, `==`, `!=`
Operador de Asignación	`=`
Separadores	`,` (coma), `;` (punto y coma)
Espacios en Blanco	`[ \t\n\r]+` (ignorados por el analizador léxico)

Tabla 1: Tokens definidos para el lenguaje SmartRule

2.5. Alfabeto del Lenguaje

El alfabeto de *SmartRule* es el conjunto finito de caracteres que acepta el analizador léxico:

$$\Sigma = \Sigma_L \cup \Sigma_D \cup \Sigma_S \cup \Sigma_W$$

donde:

- $\Sigma_L = \{\texttt{A}, \ldots, \texttt{Z},\ \texttt{a}, \ldots, \texttt{z},\ \texttt{_}\}$: letras y guion bajo.
- $\Sigma_D = \{\texttt{0}, \ldots, \texttt{9}\}$: dígitos.
- $\Sigma_S = \{\texttt{>},\ \texttt{<},\ \texttt{=},\ \texttt{!},\ \texttt{,},\ \texttt{;}\}$: símbolos de operadores y separadores.
- $\Sigma_W = \{\text{espacio},\ \text{tabulación},\ \text{salto de línea},\ \text{retorno de carro}\}$: separadores ignorados.

Cualquier otro carácter (por ejemplo `#` o `@`) produce un error léxico. Los lexemas se agrupan en las categorías de la Tabla 1 y el analizador sintáctico trabaja con los *terminales* `ID`, `NUM`, `OP_REL`, `IGUAL`, `COMA`, `PUNTO_COMA`, las palabras reservadas y el marcador de fin de entrada `$`.

2.6. Ejemplos de Programas

2.6.1. Ejemplos Válidos

Listing 1: Ejemplos válidos en SmartRule

```
WHEN temperatura > 28 AND humedad < 40 THEN encender ventilador;

WHEN luz == 0 OR movimiento == TRUE THEN encender luces, alarma = 1
    ELSE apagar luces;

WHEN NOT puerta_abierta THEN activar_alarma;

WHEN nivel_agua >= 80 THEN bomba = 1, notificar_app;

WHEN presion < 10 AND NOT emergencia THEN abrir_valvula;
```

2.6.2. Ejemplos Inválidos (Detectados por el Analizador)

Listing 2: Ejemplos inválidos en SmartRule

```
WHEN temperatura > THEN encender ventilador;        // Falta
   operando tras operador relacional
temperatura > 28 THEN encender luces;                // Falta
   palabra reservada WHEN
WHEN temperatura > 28 encender luces;                // Falta
   palabra reservada THEN
WHEN AND humedad < 40 THEN encender ventilador;      // Falta
   condición válida
WHEN temperatura > 28 THEN ;                         // Lista de
   acciones vacía
WHEN temperatura > 28 THEN accion1,, accion2;        // Coma extra
```

2.7. Extensión Creativa

En lugar de añadir elementos estéticos solo para cumplir con el requisito, decidimos implementar funcionalidades que realmente aportaran a la lógica de un sistema domótico y que se integraran de forma natural en la gramática.

Por esto, incorporamos el operador `NOT` para negar variables (asignándole la mayor precedencia), añadimos el bloque `ELSE` para manejar el flujo alternativo cuando una condición no se cumple, y permitimos la ejecución de múltiples acciones simultáneas separadas por comas.

La Tabla 2 indica dónde se refleja cada construcción: producciones y celdas de la tabla LL(1) (Sección 4) y casos de prueba (Sección 6.3).

Tabla 2: Integración de la extensión creativa en gramática, tabla LL(1) y pruebas

Construcción	**Prod.**	**Celdas de la tabla LL(1)**	**Pruebas**
Negación `NOT` (prefija, anidable)	13	$M[\mathit{NotExpr}, \texttt{NOT}]$	V3, V5, V9, L3
Cláusula alternativa `ELSE`	4, 5	$M[\mathit{ElsePart}, \texttt{ELSE}]$, $M[\mathit{ElsePart}, \texttt{PUNTO_COMA}]$	V2, V8, I9, L4
Múltiples acciones separadas por coma	23, 24	$M[\mathit{AccionTail}, \texttt{COMA}]$, $M[\mathit{AccionTail}, \texttt{ELSE}]$, $M[\mathit{AccionTail}, \texttt{PUNTO_COMA}]$	V2, V4, V10, I6, I10
Acciones con argumento, objeto y asignación	25–33	Todas las celdas de *Accion*, *AsigOrCall* y *AsigTail* (Tabla 6)	V1, V4, V6, V8

3. Gramática Libre de Contexto (GLC)

3.1. Definición Formal

Se ha diseñado una Gramática Libre de Contexto $G = (V_N, V_T, P, S)$, donde:

- V_N = {Prog, Regla, ElsePart, Cond, OrExpr, OrTail, AndExpr, AndTail, NotExpr, RelExpr, RelExprTail, Factor, ListaAcciones, AccionTail, Accion, AsigOrCall, AsigTail}
- V_T = {WHEN, THEN, ELSE, AND, OR, NOT, TRUE, FALSE, NUM, ID, OP_REL, IGUAL, COMA, PUNTO_COMA, $}
- $S = Prog$
- P = Conjunto de producciones definido a continuación

3.2. Producciones de la Gramática

Las producciones se muestran agrupadas por no terminal; en la Tabla 5 aparecen numeradas individualmente.

$$
\begin{aligned}
\textit{Prog} &\rightarrow \textit{Regla Prog} \mid \epsilon \\
\textit{Regla} &\rightarrow \texttt{WHEN}\ \textit{Cond}\ \texttt{THEN}\ \textit{ListaAcciones ElsePart}\ \texttt{PUNTO_COMA} \\
\textit{ElsePart} &\rightarrow \texttt{ELSE}\ \textit{ListaAcciones} \mid \epsilon \\
\textit{Cond} &\rightarrow \textit{OrExpr} \\
\textit{OrExpr} &\rightarrow \textit{AndExpr OrTail} \\
\textit{OrTail} &\rightarrow \texttt{OR}\ \textit{AndExpr OrTail} \mid \epsilon \\
\textit{AndExpr} &\rightarrow \textit{NotExpr AndTail} \\
\textit{AndTail} &\rightarrow \texttt{AND}\ \textit{NotExpr AndTail} \mid \epsilon \\
\textit{NotExpr} &\rightarrow \texttt{NOT}\ \textit{NotExpr} \mid \textit{RelExpr} \\
\textit{RelExpr} &\rightarrow \textit{Factor RelExprTail} \\
\textit{RelExprTail} &\rightarrow \texttt{OP_REL}\ \textit{Factor} \mid \epsilon \\
\textit{Factor} &\rightarrow \texttt{NUM} \mid \texttt{ID} \mid \texttt{TRUE} \mid \texttt{FALSE} \\
\textit{ListaAcciones} &\rightarrow \textit{Accion AccionTail} \\
\textit{AccionTail} &\rightarrow \texttt{COMA}\ \textit{Accion AccionTail} \mid \epsilon \\
\textit{Accion} &\rightarrow \texttt{ID}\ \textit{AsigOrCall} \\
\textit{AsigOrCall} &\rightarrow \texttt{IGUAL}\ \textit{Factor} \mid \texttt{ID}\ \textit{AsigTail} \mid \texttt{NUM} \mid \texttt{TRUE} \mid \texttt{FALSE} \mid \epsilon \\
\textit{AsigTail} &\rightarrow \texttt{IGUAL}\ \textit{Factor} \mid \epsilon
\end{aligned}
$$

3.3. Análisis y Transformaciones de la Gramática

3.3.1. Recursión Izquierda

Tras el análisis de las producciones, verificamos que la gramática **no presenta recursión izquierda**, ni directa ni indirecta (verificado por inspección de las producciones: ningún no terminal puede derivar una forma sentencial que comience con él mismo). Para

el caso de listas (reglas, acciones, operadores lógicos), se empleó **recursión por la derecha** mediante las reglas *Tail* (`OrTail`, `AndTail`, `AccionTail`), lo cual es idóneo para un parser LL(1) descendente recursivo.

3.3.2. Factorización por la Izquierda

Durante el análisis se detectó un **conflicto FIRST/FIRST** en la producción de acciones. La versión inicial de la gramática era:

$$\begin{aligned} \textit{Accion} &\rightarrow \textit{ID AsigOrCall} \mid \textit{ID ID IGUAL Factor} \\ \textit{AsigOrCall} &\rightarrow \textit{IGUAL Factor} \mid \textit{Factor} \mid \epsilon \end{aligned}$$

Las dos alternativas de *Accion* comienzan con `ID`, por lo que con un solo token de anticipación el parser no puede elegir entre ellas. Además, $\textit{AsigOrCall} \rightarrow \textit{Factor}$ también puede derivar `ID`, de modo que una entrada como `encender luces` admitía más de una derivación. Por lo tanto esa versión **no era LL(1)**.

Se resolvió **factorizando por la izquierda** el prefijo común `ID` y extrayendo la parte opcional de la asignación en un nuevo no terminal, *AsigTail*:

$$\begin{aligned} \textit{Accion} &\rightarrow \texttt{ID}\ \textit{AsigOrCall} \\ \textit{AsigOrCall} &\rightarrow \texttt{IGUAL}\ \textit{Factor} \mid \texttt{ID}\ \textit{AsigTail} \mid \texttt{NUM} \mid \texttt{TRUE} \mid \texttt{FALSE} \mid \epsilon \\ \textit{AsigTail} &\rightarrow \texttt{IGUAL}\ \textit{Factor} \mid \epsilon \end{aligned}$$

El lenguaje generado no cambia; las cuatro formas de acción siguen siendo válidas:

- Asignación directa, `bomba = 1`: $\textit{AsigOrCall} \rightarrow \textit{IGUAL Factor}$.
- Llamada sin argumento, `activar_alarma`: $\textit{AsigOrCall} \rightarrow \epsilon$.
- Llamada con argumento, `encender luces` o `alarma 5`: $\textit{AsigOrCall} \rightarrow \textit{ID AsigTail}$ con $\textit{AsigTail} \rightarrow \epsilon$, o $\textit{AsigOrCall} \rightarrow \textit{NUM}$.
- Asignación con objeto, `encender luces = 100`: $\textit{AsigOrCall} \rightarrow \textit{ID AsigTail}$ con $\textit{AsigTail} \rightarrow \textit{IGUAL Factor}$.

3.3.3. Ambigüedad y Precedencia

La gramática es **no ambigua**: toda gramática LL(1) lo es, y en la Sección 4.3 se demuestra que ésta cumple la condición LL(1). La precedencia de operadores se garantiza mediante la jerarquía de no terminales:

$$NOT > AND > OR$$

Las comparaciones relacionales (`OP_REL`) se evalúan entre dos factores y no son encadenables, tal como se define en *RelExpr* (el caso `I8` de la Tabla 9 lo comprueba).

4. Conjuntos FIRST, FOLLOW y Tabla LL(1)

4.1. Conjuntos FIRST

El conjunto $FIRST(\alpha)$ contiene todos los terminales con los cuales puede comenzar cualquier cadena derivable desde α; incluye ϵ si α puede derivar la cadena vacía.

No Terminal	**FIRST**
Prog	{WHEN, ϵ}
Regla	{WHEN}
ElsePart	{ELSE, ϵ}
Cond	{NOT, TRUE, FALSE, NUM, ID}
OrExpr	{NOT, TRUE, FALSE, NUM, ID}
OrTail	{OR, ϵ}
AndExpr	{NOT, TRUE, FALSE, NUM, ID}
AndTail	{AND, ϵ}
NotExpr	{NOT, TRUE, FALSE, NUM, ID}
RelExpr	{TRUE, FALSE, NUM, ID}
RelExprTail	{OP_REL, ϵ}
Factor	{TRUE, FALSE, NUM, ID}
ListaAcciones	{ID}
AccionTail	{COMA, ϵ}
Accion	{ID}
AsigOrCall	{TRUE, FALSE, NUM, ID, IGUAL, ϵ}
AsigTail	{IGUAL, ϵ}

Tabla 3: Conjuntos FIRST calculados

4.2. Conjuntos FOLLOW

El conjunto $FOLLOW$(A) contiene todos los terminales que pueden aparecer inmediatamente después del no-terminal A en alguna derivación sentencial. Algunos casos que conviene justificar:

- `AND` $\in FOLLOW(NotExpr)$, $FOLLOW(RelExpr)$ y $FOLLOW(RelExprTail)$, porque en $AndExpr \rightarrow NotExpr\ AndTail$ lo que sigue a $NotExpr$ es $AndTail$, cuyo FIRST contiene `AND`.
- $FOLLOW(Factor)$ reúne `OP_REL` (por $RelExprTail$), los seguidores de $RelExpr$ (`AND`, `OR`, `THEN`) y los seguidores de $AsigOrCall$ y $AsigTail$ (`COMA`, `ELSE`, `PUNTO_COMA`), porque $Factor$ puede ser el último símbolo de esas producciones.
- $FOLLOW(ElsePart) =$ {`PUNTO_COMA`}, ya que $ElsePart$ cierra la regla justo antes de ;.

No Terminal	FOLLOW
Prog	{$}
Regla	{WHEN, $}
ElsePart	{PUNTO_COMA}
Cond	{THEN}
OrExpr	{THEN}
OrTail	{THEN}
AndExpr	{THEN, OR}
AndTail	{THEN, OR}
NotExpr	{THEN, AND, OR}
RelExpr	{THEN, AND, OR}
RelExprTail	{THEN, AND, OR}
Factor	{THEN, ELSE, AND, OR, OP_REL, COMA, PUNTO_COMA}
ListaAcciones	{ELSE, PUNTO_COMA}
AccionTail	{ELSE, PUNTO_COMA}
Accion	{ELSE, COMA, PUNTO_COMA}
AsigOrCall	{ELSE, COMA, PUNTO_COMA}
AsigTail	{ELSE, COMA, PUNTO_COMA}

Tabla 4: Conjuntos FOLLOW calculados

4.3. Tabla LL(1) y Verificación de Conflictos

Para cada producción $A \to \alpha$ se define su *conjunto de selección*: $FIRST(\alpha) \setminus \{\epsilon\}$, más $FOLLOW(A)$ cuando $\epsilon \in FIRST(\alpha)$. La entrada $M[A, a]$ contiene la producción cuyo conjunto de selección incluye al terminal a [2, Cap. 5]. La Tabla 5 numera las producciones y entrega sus conjuntos de selección, y la Tabla 6 muestra la tabla predictiva resultante.

Tabla 5: Producciones numeradas y conjuntos de selección

N.º	Producción	Selección
1	*Prog* → *Regla Prog*	{WHEN}
2	*Prog* → ϵ	{$}
3	*Regla* → `WHEN` *Cond* `THEN` *ListaAcciones ElsePart* `PUNTO_COMA`	{WHEN}
4	*ElsePart* → `ELSE` *ListaAcciones*	{ELSE}
5	*ElsePart* → ϵ	{PUNTO_COMA}
6	*Cond* → *OrExpr*	{NOT, TRUE, FALSE, NUM, ID}
7	*OrExpr* → *AndExpr OrTail*	{NOT, TRUE, FALSE, NUM, ID}
8	*OrTail* → `OR` *AndExpr OrTail*	{OR}
9	*OrTail* → ϵ	{THEN}
10	*AndExpr* → *NotExpr AndTail*	{NOT, TRUE, FALSE, NUM, ID}
11	*AndTail* → `AND` *NotExpr AndTail*	{AND}
12	*AndTail* → ϵ	{THEN, OR}
13	*NotExpr* → `NOT` *NotExpr*	{NOT}

N.º	**Producción**	**Selección**
14	*NotExpr* → *RelExpr*	{TRUE, FALSE, NUM, ID}
15	*RelExpr* → *Factor RelExprTail*	{TRUE, FALSE, NUM, ID}
16	*RelExprTail* → `OP_REL` *Factor*	{OP_REL}
17	*RelExprTail* → ϵ	{THEN, AND, OR}
18	*Factor* → `NUM`	{NUM}
19	*Factor* → `ID`	{ID}
20	*Factor* → `TRUE`	{TRUE}
21	*Factor* → `FALSE`	{FALSE}
22	*ListaAcciones* → *Accion AccionTail*	{ID}
23	*AccionTail* → `COMA` *Accion AccionTail*	{COMA}
24	*AccionTail* → ϵ	{ELSE, PUNTO_COMA}
25	*Accion* → `ID` *AsigOrCall*	{ID}
26	*AsigOrCall* → `IGUAL` *Factor*	{IGUAL}
27	*AsigOrCall* → `ID` *AsigTail*	{ID}
28	*AsigOrCall* → `NUM`	{NUM}
29	*AsigOrCall* → `TRUE`	{TRUE}
30	*AsigOrCall* → `FALSE`	{FALSE}
31	*AsigOrCall* → ϵ	{ELSE, COMA, PUNTO_COMA}
32	*AsigTail* → `IGUAL` *Factor*	{IGUAL}
33	*AsigTail* → ϵ	{ELSE, COMA, PUNTO_COMA}

M	WHEN	THEN	ELSE	AND	OR	NOT	TRUE	FALSE	NUM	ID	OP_REL	IGUAL	COMA	PUNTO_COMA	$
Prog	1														2
Regla	3														
ElsePart			4											5	
Cond						6	6	6	6	6					
OrExpr						7	7	7	7	7					
OrTail		9			8										
AndExpr						10	10	10	10	10					
AndTail		12		11	12										
NotExpr						13	14	14	14	14					
RelExpr							15	15	15	15					
RelExprTail		17		17	17						16				
Factor							20	21	18	19					
ListaAcciones										22					
AccionTail			24										23	24	
Accion										25					
AsigOrCall			31				29	30	28	27		26	31	31	
AsigTail			33									32	33	33	

Tabla 6: Tabla predictiva LL(1): cada celda contiene el número de la producción de la Tabla 5; las celdas vacías indican error sintáctico

Una gramática es LL(1) si y solo si, para cada no terminal con varias alternativas, sus conjuntos de selección son disjuntos de a pares. Se verificó con el script `first_follow.py` (carpeta `src/utils/`) y se concluye:

- La gramática cumple las condiciones de la clase **LL(1)**: ninguna celda $M[A, a]$ recibe más de una producción.
- **No hay conflictos** FIRST/FIRST (alternativas que comparten un terminal inicial) ni FIRST/FOLLOW (una alternativa con ϵ cuyo FOLLOW coincide con el FIRST de otra alternativa).
- La tabla predictiva es determinista para todos los pares $(NoTerminal, Terminal)$.
- El conflicto FIRST/FIRST de la versión inicial (Sección 3.3.2) quedó eliminado: $M[Accion, ID]$ contiene únicamente la producción 25.

El análisis completo puede reproducirse ejecutando:

```
python -m src.utils.first_follow
```

5. Diseño e Implementación

5.1. Arquitectura del Proyecto

Se ha adoptado una **arquitectura limpia y modular**, separando responsabilidades por capas con el objetivo de facilitar la mantenibilidad, extensibilidad y legibilidad del código.

```
proyecto_teoria1/
|-- src/
|   |-- lexer/      # Analizador Léxico
|   |-- parser/     # Analizador Sintáctico LL(1) Descendente
   Recursivo
|   |-- ast/        # Nodos del Árbol Sintáctico Abstracto
|   `-- utils/      # Cálculo de FIRST/FOLLOW, visualización AST
|-- tests/          # Suite de pruebas unitarias e integradas
|-- ejemplos/       # Archivos .sr con casos válidos e inválidos
|-- docs/           # Documentación, gramática e informe LaTeX
|-- main.py         # Punto de entrada CLI (interactivo y por
   archivo)
`-- README.md       # Guía de ejecución y documentación
```

Figura 1: Estructura del proyecto con arquitectura limpia

5.2. Analizador Léxico

El analizador léxico fue tomado como base del cuaderno `anlizador_lexico1.ipynb` provisto por el docente, y posteriormente **extendido** para dar soporte a los requerimientos del lenguaje *SmartRule*.

Características implementadas:

- Implementación mediante `re.finditer` con patrones nombrados [3, sección *Writing a Tokenizer*].
- Reconocimiento de operadores relacionales compuestos (`>=`, `<=`, `==`, `!=`).
- Soporte para `=` (`IGUAL`) y `,` (`COMA`).
- Clasificación automática de palabras reservadas (`WHEN`, `THEN`, `ELSE`, `AND`, `OR`, `NOT`, `TRUE`, `FALSE`).
- Ignorado de espacios, tabulaciones y saltos de línea.
- Detección de símbolos no reconocidos con excepción `LexerError`.
- Adición automática del token `EOF` ($) para facilitar el análisis sintáctico.

5.3. Analizador Sintáctico LL(1) Descendente Recursivo

Implementamos el analizador sintáctico completamente desde cero. Aunque existía la opción de usar generadores automáticos como ANTLR o Yacc, preferimos programarlo a mano para cumplir con la rúbrica y entender realmente el funcionamiento interno del análisis predictivo.

En la arquitectura del código, creamos un método `parse_*()` para cada no-terminal de la gramática, configurando el parser para que decida su camino leyendo un solo token de anticipación, sin usar *backtracking*. Esta estructura nos permitió manejar de forma limpia la recursión por la derecha, el bloque `ELSE` y los distintos tipos de asignaciones.

5.4. Árbol Sintáctico Abstracto (AST)

Se construyó un AST explícito con el fin de representar la estructura semántica del programa de forma clara y jerárquica.

Clase Nodo	Propósito
`Programa`	Raíz del árbol. Contiene lista de reglas.
`Regla`	Representa `WHEN Cond THEN ... ELSE ... ;`
`BinOp`	Operadores binarios (`AND`, `OR`, `==`, `>`, `<`, etc.)
`UnOp`	Operador unario (`NOT`)
`Literal`	Valores numéricos o booleanos (`NUM`, `TRUE`, `FALSE`)
`Identificador`	Variables, sensores, actuadores o acciones
`ListaAcciones`	Conjunto de acciones a ejecutar
`Asignacion`	Acción de asignación (`variable = valor`)
`LlamadaAccion`	Llamado a acción con o sin argumento

Tabla 7: Nodos del AST implementados

5.5. Manejo de Errores

Diseñamos el sistema de errores con la idea de que los mensajes guíen la corrección del código, en lugar de solo lanzar una excepción genérica que detenga el programa. Esto resultó ser muy útil durante nuestra propia fase de depuración.

Si se ingresa un carácter inválido, el analizador léxico (`LexerError`) indica la línea y columna exacta del problema. Por el lado sintáctico, el `ParserError` detalla qué token se esperaba y cuál se encontró en esa posición. Además, agregamos una opción (`-e`) para que el analizador no se detenga en el primer error y reporte todos los fallos encontrados en una sola pasada.

6. Pruebas, Ejecución y Resultados

6.1. Ejecución del Analizador

El proyecto cuenta con una interfaz CLI intuitiva que permite analizar archivos o trabajar en modo interactivo.

6.1.1. Modo Interactivo

```
python main.py
```

6.1.2. Analizar archivo con salida completa

```
python main.py -f ejemplos/validos_1.sr -t -a
```

Donde `-t` muestra los tokens generados y `-a` muestra el AST construido.

6.1.3. Verificar FIRST, FOLLOW y Tabla LL(1)

```
python -m src.utils.first_follow
```

6.2. Suite de Pruebas Automatizadas

Se diseñó una batería de pruebas completa para validar tanto el análisis léxico como el sintáctico. Las pruebas se encuentran organizadas en la carpeta `tests/`.

Archivo	Cantidad	Objetivo
`test_lexer.py`	7	Tokens, palabras reservadas, operadores, `=`, `,`, errores léxicos con su formato (línea y columna) y rechazo de paréntesis.
`test_parser.py`	17	Reglas simples/complejas, `AND/OR/NOT`, `ELSE`, múltiples acciones, asignaciones, programa vacío, `NOT` no válido como operando, detección de errores sintácticos.
`test_integracion.py`	6	Casos completos por dominio (casa inteligente, dron, riego agrícola), todos los relacionales, `NOT` anidado, precedencia `AND`/`OR`.
`test_casos_informe.py`	25	Los 25 casos documentados en la Tabla 9 (10 válidos, 10 inválidos, 5 límite), con verificación del mensaje de error.
`test_errores_tarea4.py`	7	Modo pánico (varios errores), línea/columna en entradas multilínea y mensajes de error.
`test_gramatica_ll1.py`	4	Ausencia de conflictos LL(1), detección de conflicto en una gramática de juguete, FIRST/FOLLOW de *Accion* y de *NotExpr*.
`test_ast_forma.py`	1	Forma del AST y precedencia de `NOT` sobre `AND`.
`test_casos.py`	1	Recorre los 32 casos de `tests/casos.py` (válidos, inválidos y límite).
Total	**68**	Cobertura amplia y variada

Tabla 8: Distribución de pruebas automatizadas

6.3. Casos de Prueba Documentados

Además de la suite automatizada, se documentan 25 casos: 10 sintácticamente válidos (`V1`–`V10`), 10 inválidos (`I1`–`I10`) y 5 casos límite (`L1`–`L5`). La columna *Obtenido* muestra el resultado del analizador, con la columna donde se detecta el error. En los 25 casos el resultado obtenido coincide con el esperado.

Tabla 9: Casos de prueba documentados (10 válidos, 10 inválidos y 5 límite)

N.º	Entrada	Esperado	Obtenido	Observación
V1	`WHEN temperatura > 28 AND humedad < 40 THEN encender ventilador;`	Acepta	Aceptada	Condición compuesta con AND; acción con argumento.
V2	`WHEN luz == 0 OR movimiento == TRUE THEN encender luces, alarma = 1 ELSE apagar luces;`	Acepta	Aceptada	OR, múltiples acciones, asignación y cláusula ELSE.

continúa en la página siguiente

N.º	Entrada	Esperado	Obtenido	Observación
V3	`WHEN NOT puerta_abierta THEN activar_alarma;`	Acepta	Aceptada	Negación NOT sobre un factor; acción sin argumento.
V4	`WHEN nivel_agua >= 80 THEN bomba = 1, notificar_app;`	Acepta	Aceptada	Operador >=; asignación seguida de llamada.
V5	`WHEN presion < 10 AND NOT emergencia THEN abrir_valvula;`	Acepta	Aceptada	NOT dentro de una conjunción.
V6	`WHEN humedad <= 30 THEN encender riego = 100;`	Acepta	Aceptada	Asignación con objeto (ID ID = valor).
V7	`WHEN movimiento == TRUE THEN activar alarma; WHEN luz < 5 THEN encender luces;`	Acepta	Aceptada	Dos reglas en la misma entrada.
V8	`WHEN a == 1 OR b == 2 AND c != 3 THEN x = 1 ELSE x = 0;`	Acepta	Aceptada	Precedencia AND sobre OR; operador !=; ELSE con asignación.
V9	`WHEN NOT NOT activo THEN apagar;`	Acepta	Aceptada	NOT anidado.
V10	`WHEN bateria < 20 THEN volver_base, aterrizar, notificar operador;`	Acepta	Aceptada	Tres acciones separadas por coma.
I1	`WHEN temperatura > THEN encender ventilador;`	Rechaza	Rechazada (col. 20): se esperaba un factor, se encontró `THEN`	Falta operando tras el operador relacional.
I2	`temperatura > 28 THEN encender luces;`	Rechaza	Rechazada (col. 1): se esperaba la palabra reservada WHEN o el fin de la entrada, se encontró `temperatura`	Falta la palabra reservada WHEN.
I3	`WHEN temperatura > 28 encender luces;`	Rechaza	Rechazada (col. 23): se esperaba THEN, se encontró `encender`	Falta la palabra reservada THEN.
I4	`WHEN AND humedad < 40 THEN encender ventilador;`	Rechaza	Rechazada (col. 6): se esperaba un factor, se encontró `AND`	Condición que empieza con operador lógico.
I5	`WHEN temperatura > 28 THEN ;`	Rechaza	Rechazada (col. 28): se esperaba el nombre de una acción (ID), se encontró ;	Lista de acciones vacía.
I6	`WHEN temperatura > 28 THEN accion1,, accion2;`	Rechaza	Rechazada (col. 36): se esperaba el nombre de una acción (ID), se encontró ,	Coma duplicada.
I7	`WHEN temperatura > 28 THEN encender ventilador`	Rechaza	Rechazada (col. 47): se esperaba ; al final de la regla, se encontró fin de la entrada	Falta el punto y coma final.
I8	`WHEN temperatura > 28 > 30 THEN encender ventilador;`	Rechaza	Rechazada (col. 23): se esperaba THEN, se encontró >	Comparación encadenada no permitida.
I9	`WHEN luz == 0 THEN encender luces ELSE ;`	Rechaza	Rechazada (col. 40): se esperaba el nombre de una acción (ID), se encontró ;	ELSE sin acciones.

continúa en la página siguiente

N.º	Entrada	Esperado	Obtenido	Observación
I10	`WHEN luz == 0 THEN encender luces, ;`	Rechaza	Rechazada (col. 36): se esperaba el nombre de una acción (ID), se encontró ;	Coma final sin acción.
L1	*(entrada vacía)*	Acepta	Aceptada	Programa vacío: Prog deriva la cadena vacía.
L2	`WHEN x THEN a;`	Acepta	Aceptada	Regla mínima: condición con un solo factor.
L3	`WHEN NOT NOT NOT x THEN a;`	Acepta	Aceptada	Tres NOT consecutivos.
L4	`WHEN x THEN a; WHEN`	Rechaza	Rechazada (col. 20): se esperaba un factor, se encontró fin de la entrada	Entrada truncada: termina en medio de una regla.
L5	`WHEN x # 3 THEN a;`	Rechaza	Rechazada (col. 8): error léxico, símbolo no reconocido #	Carácter fuera del alfabeto (error léxico).

6.4. Resultados de las Pruebas

Las pruebas automatizadas fueron ejecutadas con `unittest` desde el terminal integrado de Visual Studio Code (Figura 2, Anexo E).

Resultado: `Ran 68 tests ... OK` – todas las pruebas pasaron exitosamente.

6.5. Ejemplo de Salida del Analizador

Al analizar el archivo `ejemplos/validos_1.sr` (capturas en las Figuras 3 y 4, Anexo E), el sistema confirma el éxito del análisis:

Listing 3: Salida exitosa del analizador

```
============================================================
ANALIZADOR SINTACTICO SMART RULE (LL(1))
============================================================

Entrada:
WHEN temperatura > 28 AND humedad < 40 THEN encender ventilador;
...

[OK] ANALISIS LEXICO EXITOSO
[OK] ANALISIS SINTACTICO EXITOSO

Se reconocieron 5 regla(s) valida(s).
```

El AST completo de la primera regla del archivo (`V1`) es el siguiente: la condición es una conjunción de dos comparaciones y la acción es una llamada con un argumento, sin cláusula `ELSE`.

Listing 4: AST de la regla `WHEN temperatura >28 AND humedad <40 THEN encender ventilador;`

```
+-- Programa
    +-- Regla
        +-- Condicion:
```

```
            +-- BinOp(AND)
                +-- BinOp(>)
                    +-- Identificador(temperatura)
                    +-- Literal(NUM=28)
                +-- BinOp(<)
                    +-- Identificador(humedad)
                    +-- Literal(NUM=40)
        +-- AccionesTHEN:
            +-- ListaAcciones
                +-- LlamadaAccion(encender)
                    +-- argumento:
                        +-- Identificador(ventilador)
        +-- AccionesELSE: None
```

Al analizar entradas inválidas, el sistema reporta el error indicando lo esperado, lo encontrado y su posición:

Listing 5: Detección de errores léxicos y sintácticos

```
Entrada: WHEN temperatura > THEN encender ventilador;
Error sintáctico (token 4, línea 1, columna 20): se esperaba un
   factor (NUM, ID, TRUE o FALSE), se encontró 'THEN'

Entrada: WHEN temperatura > 28 encender luces;
Error sintáctico (token 5, línea 1, columna 23): se esperaba THEN,
   se encontró 'encender'

Entrada: WHEN temperatura > 28 THEN encender ventilador
Error sintáctico (token 8, línea 1, columna 47): se esperaba ';' al
    final de la regla, se encontró fin de la entrada

Entrada: WHEN x # 3 THEN a;
Error léxico (línea 1, columna 8): símbolo no reconocido '#'
```

7. Conclusiones

Al terminar el proyecto, podemos decir que logramos lo que nos propusimos: diseñar e implementar un analizador sintáctico LL(1) descendente recursivo que funciona para *SmartRule*. Más allá del resultado, el proceso nos sirvió para entender de verdad cómo se conectan la teoría y la práctica.

- **Una gramática que cumple lo que prometía:** La GLC que diseñamos no es ambigua, respeta la precedencia de los operadores y es LL(1). En el camino encontramos un conflicto FIRST/FIRST en las acciones, que resolvimos factorizando por la izquierda. Fue una buena experiencia, porque vimos en un caso real por qué importan estas condiciones.
- **Análisis formal completo:** Calculamos y justificamos los conjuntos *FIRST* y *FOLLOW* y con ellos armamos la tabla LL(1), que quedó sin conflictos.
- **Implementación propia:** Programamos el parser desde cero, sin generadores automáticos, y extendimos el analizador léxico a partir del cuaderno base que entregó el profesor.
- **Árbol y errores claros:** El AST muestra de forma ordenada la estructura de cada regla, y el sistema reporta los errores léxicos y sintácticos indicando qué se esperaba, qué se encontró y dónde, lo que ayuda bastante a corregirlos.
- **Pruebas y orden del código:** Las 68 pruebas automatizadas, que incluyen los 25 casos documentados (10 válidos, 10 inválidos y 5 límite), pasaron correctamente. Además, organizamos el código con una arquitectura modular y dejamos documentación (`README.md`, gramática formal, este informe en LaTeX y ejemplos) para que cualquiera pueda reproducir el trabajo.

Este proyecto nos permitió aplicar los contenidos del curso en algo concreto y nos dejó una base sólida para seguir mejorando el lenguaje en el futuro.

8. Recomendaciones y Trabajo Futuro

Como trabajo futuro para seguir mejorando el lenguaje, consideramos que el siguiente paso lógico sería incorporar soporte para paréntesis, lo que permitiría agrupar condiciones complejas con mayor libertad. También sería útil sumar una etapa de análisis semántico para validar la existencia de variables y comprobar los tipos de datos antes de evaluar la regla.

A más largo plazo, nos gustaría traducir estas reglas a código ejecutable dentro de un simulador IoT, además de desarrollar una interfaz gráfica para visualizar el árbol sintáctico directamente, sin depender de exportar archivos a `graphviz`.

9. Referencias Bibliográficas

Referencias

[1] Brena, R. (2003). *Autómatas y Lenguajes: Un enfoque de diseño.* Tecnológico de Monterrey.

[2] Navarrete Sánchez, I., Cárdenas Viedma, M. A., Sánchez Álvarez, D., Botía Blaya, J. A., Marín Morales, R. & Martínez Béjar, R. (s. f.). *Teoría de Autómatas y Lenguajes Formales.* Departamento de Ingeniería de la Información y las Comunicaciones, Universidad de Murcia.

[3] Python Software Foundation. (s. f.). *re — Regular expression operations.* Python 3 Documentation. Recuperado el 30 de octubre de 2026, de `https://docs.python.org/3/library/re.html`

Declaración de uso de inteligencia artificial

Se utilizó **Claude** (Anthropic) como apoyo durante el proyecto para revisar la coherencia entre la gramática, los conjuntos *FIRST* y *FOLLOW*, la tabla LL(1), las pruebas y el informe, y para proponer correcciones puntuales al código y a la documentación. También se utilizó **OpenCode** para aplicar ajustes al código del repositorio. El diseño de la gramática, la implementación del analizador y las pruebas fueron desarrollados por el grupo, que revisó, comprendió y es responsable de toda la solución presentada.

10. Anexos

10.1. Anexo A: Repositorio del Proyecto

El código fuente completo, pruebas, ejemplos y documentación se encuentran en el repositorio GitHub del grupo:

`https://github.com/saezjose/Implementacion_de_un_analizador_sintactico_Proyecto_Teoria_Computacion`

10.2. Anexo B: Instrucciones para Compilar y Ejecutar

10.2.1. Requisitos

- Python 3.10 o superior
- Sistema Operativo: Windows, Linux o macOS
- No requiere dependencias externas (solo librería estándar; `pytest` es opcional)

10.2.2. Ejecución

Listing 6: Comandos de ejecución

```
# Clonar repositorio
git clone https://github.com/saezjose/
   Implementacion_de_un_analizador_sintactico_Proyecto_Teoria_Computacion
   .git
cd
   Implementacion_de_un_analizador_sintactico_Proyecto_Teoria_Computacion


# Modo interactivo
python main.py

# Analizar archivo de ejemplos
python main.py -f ejemplos/validos_1.sr

# Analizar con tokens y AST
python main.py -f ejemplos/validos_1.sr -t -a

# Ver conjuntos FIRST, FOLLOW y tabla LL(1)
python -m src.utils.first_follow

# Ejecutar todas las pruebas
python -m pytest tests -v
# o
python -m unittest discover -s tests -v
```

10.3. Anexo C: Gramática Completa en Formato BNF

Listing 7: Gramática BNF completa de SmartRule

```
<Prog>           ::= <Regla> <Prog> | ϵ
<Regla>          ::= WHEN <Cond> THEN <ListaAcciones> <ElsePart>
   PUNTO_COMA
<ElsePart>       ::= ELSE <ListaAcciones> | ϵ
<Cond>           ::= <OrExpr>
<OrExpr>         ::= <AndExpr> <OrTail>
<OrTail>         ::= OR <AndExpr> <OrTail> | ϵ
<AndExpr>        ::= <NotExpr> <AndTail>
<AndTail>        ::= AND <NotExpr> <AndTail> | ϵ
<NotExpr>        ::= NOT <NotExpr> | <RelExpr>
<RelExpr>        ::= <Factor> <RelExprTail>
<RelExprTail>    ::= OP_REL <Factor> | ϵ
<Factor>         ::= NUM | ID | TRUE | FALSE
<ListaAcciones>  ::= <Accion> <AccionTail>
<AccionTail>     ::= COMA <Accion> <AccionTail> | ϵ
<Accion>         ::= ID <AsigOrCall>
<AsigOrCall>     ::= IGUAL <Factor> | ID <AsigTail> | NUM | TRUE |
   FALSE | ϵ
<AsigTail>       ::= IGUAL <Factor> | ϵ
```

10.4. Anexo D: Derivación por la Izquierda

10.4.1. Objetivo

Demostrar la derivación por la izquierda para la siguiente sentencia válida del lenguaje *SmartRule*:

```
WHEN temperatura > 28 AND humedad < 40 THEN encender ventilador;
```

10.4.2. Gramática utilizada

Se utiliza la gramática LL(1) definida en el Anexo C; las producciones están numeradas en la Tabla 5.

10.4.3. Derivación por la izquierda

Símbolo inicial: *Prog*. En cada paso se expande el no terminal más a la izquierda; los no terminales se escriben en cursiva y los terminales en fuente monoespaciada (las producciones ϵ simplemente eliminan el no terminal). La derivación se generó siguiendo la tabla LL(1) con un token de anticipación.

1. *Prog* $\rightarrow$ *Regla Prog* (producción 1)
   $\underset{lm}{\Rightarrow}$ *Regla Prog*
2. *Regla* $\rightarrow$ `WHEN` *Cond* `THEN` *ListaAcciones ElsePart* `PUNTO_COMA` (producción 3)
   $\underset{lm}{\Rightarrow}$ `WHEN` *Cond* `THEN` *ListaAcciones ElsePart* `PUNTO_COMA` *Prog*
3. *Cond* $\rightarrow$ *OrExpr* (producción 6)
   $\underset{lm}{\Rightarrow}$ `WHEN` *OrExpr* `THEN` *ListaAcciones ElsePart* `PUNTO_COMA` *Prog*
4. *OrExpr* $\rightarrow$ *AndExpr OrTail* (producción 7)
   $\underset{lm}{\Rightarrow}$ `WHEN` *AndExpr OrTail* `THEN` *ListaAcciones ElsePart* `PUNTO_COMA` *Prog*
5. *AndExpr* $\rightarrow$ *NotExpr AndTail* (producción 10)
   $\underset{lm}{\Rightarrow}$ `WHEN` *NotExpr AndTail OrTail* `THEN` *ListaAcciones ElsePart* `PUNTO_COMA` *Prog*
6. *NotExpr* $\rightarrow$ *RelExpr* (producción 14)
   $\underset{lm}{\Rightarrow}$ `WHEN` *RelExpr AndTail OrTail* `THEN` *ListaAcciones ElsePart* `PUNTO_COMA` *Prog*
7. *RelExpr* $\rightarrow$ *Factor RelExprTail* (producción 15)
   $\underset{lm}{\Rightarrow}$ `WHEN` *Factor RelExprTail AndTail OrTail* `THEN` *ListaAcciones ElsePart* `PUNTO_COMA` *Prog*
8. *Factor* $\rightarrow$ `ID` (producción 19)
   $\underset{lm}{\Rightarrow}$ `WHEN ID` *RelExprTail AndTail OrTail* `THEN` *ListaAcciones ElsePart* `PUNTO_COMA` *Prog*
9. *RelExprTail* $\rightarrow$ `OP_REL` *Factor* (producción 16)
   $\underset{lm}{\Rightarrow}$ `WHEN ID OP_REL` *Factor AndTail OrTail* `THEN` *ListaAcciones ElsePart* `PUNTO_COMA` *Prog*
10. *Factor* $\rightarrow$ `NUM` (producción 18)
    $\underset{lm}{\Rightarrow}$ `WHEN ID OP_REL NUM` *AndTail OrTail* `THEN` *ListaAcciones ElsePart* `PUNTO_COMA` *Prog*
11. *AndTail* $\rightarrow$ `AND` *NotExpr AndTail* (producción 11)
    $\underset{lm}{\Rightarrow}$ `WHEN ID OP_REL NUM AND` *NotExpr AndTail OrTail* `THEN` *ListaAcciones ElsePart* `PUNTO_COMA` *Prog*

12. *NotExpr* → *RelExpr* (producción 14)
    $\underset{lm}{\Rightarrow}$ `WHEN ID OP_REL NUM AND` *RelExpr AndTail OrTail* `THEN` *ListaAcciones ElsePart* `PUNTO_COMA` *Prog*

13. *RelExpr* → *Factor RelExprTail* (producción 15)
    $\underset{lm}{\Rightarrow}$ `WHEN ID OP_REL NUM AND` *Factor RelExprTail AndTail OrTail* `THEN` *ListaAcciones ElsePart* `PUNTO_COMA` *Prog*

14. *Factor* → `ID` (producción 19)
    $\underset{lm}{\Rightarrow}$ `WHEN ID OP_REL NUM AND ID` *RelExprTail AndTail OrTail* `THEN` *ListaAcciones ElsePart* `PUNTO_COMA` *Prog*

15. *RelExprTail* → `OP_REL` *Factor* (producción 16)
    $\underset{lm}{\Rightarrow}$ `WHEN ID OP_REL NUM AND ID OP_REL` *Factor AndTail OrTail* `THEN` *ListaAcciones ElsePart* `PUNTO_COMA` *Prog*

16. *Factor* → `NUM` (producción 18)
    $\underset{lm}{\Rightarrow}$ `WHEN ID OP_REL NUM AND ID OP_REL NUM` *AndTail OrTail* `THEN` *ListaAcciones ElsePart* `PUNTO_COMA` *Prog*

17. *AndTail* → ϵ (producción 12)
    $\underset{lm}{\Rightarrow}$ `WHEN ID OP_REL NUM AND ID OP_REL NUM` *OrTail* `THEN` *ListaAcciones ElsePart* `PUNTO_COMA` *Prog*

18. *OrTail* → ϵ (producción 9)
    $\underset{lm}{\Rightarrow}$ `WHEN ID OP_REL NUM AND ID OP_REL NUM THEN` *ListaAcciones ElsePart* `PUNTO_COMA` *Prog*

19. *ListaAcciones* → *Accion AccionTail* (producción 22)
    $\underset{lm}{\Rightarrow}$ `WHEN ID OP_REL NUM AND ID OP_REL NUM THEN` *Accion AccionTail ElsePart* `PUNTO_COMA` *Prog*

20. *Accion* → `ID` *AsigOrCall* (producción 25)
    $\underset{lm}{\Rightarrow}$ `WHEN ID OP_REL NUM AND ID OP_REL NUM THEN ID` *AsigOrCall AccionTail ElsePart* `PUNTO_COMA` *Prog*

21. *AsigOrCall* → `ID` *AsigTail* (producción 27)
    $\underset{lm}{\Rightarrow}$ `WHEN ID OP_REL NUM AND ID OP_REL NUM THEN ID ID` *AsigTail AccionTail ElsePart* `PUNTO_COMA` *Prog*

22. *AsigTail* → ϵ (producción 33)
    $\underset{lm}{\Rightarrow}$ `WHEN ID OP_REL NUM AND ID OP_REL NUM THEN ID ID` *AccionTail ElsePart* `PUNTO_COMA` *Prog*

23. *AccionTail* → ϵ (producción 24)
    $\underset{lm}{\Rightarrow}$ `WHEN ID OP_REL NUM AND ID OP_REL NUM THEN ID ID` *ElsePart* `PUNTO_COMA` *Prog*

24. *ElsePart* → ϵ (producción 5)
    $\underset{lm}{\Rightarrow}$ `WHEN ID OP_REL NUM AND ID OP_REL NUM THEN ID ID PUNTO_COMA` *Prog*

25. *Prog* → ϵ (producción 2)
    $\underset{lm}{\Rightarrow}$ `WHEN ID OP_REL NUM AND ID OP_REL NUM THEN ID ID PUNTO_COMA`

10.4.4. Cadena derivada

La forma sentencial final es la secuencia de terminales

```
WHEN ID OP_REL NUM AND ID OP_REL NUM THEN ID ID PUNTO_COMA
```

que corresponde, en el mismo orden, a los lexemas

```
WHEN temperatura > 28 AND humedad < 40 THEN encender ventilador ;
```

Por consiguiente, se demuestra que la sentencia pertenece al lenguaje generado por la gramática $G_{SmartRule}$ mediante derivación por la izquierda.

10.5. Anexo E: Demostración Funcional

Capturas de pantalla de la ejecución del proyecto.

vscode_pruebas_real.png

Figura 2: Ejecución de pruebas en el terminal integrado de VS Code

vscode_ejecucion_real0.png

Figura 3: Ejecución del analizador en el terminal integrado de VS Code

vscode_ejecucion_real1.png

Figura 4: Ejecución del analizador: continuación del AST

vscode_errores_real.png

Figura 5: Detección de errores sintácticos en el terminal integrado de VS Code

vscode_ll1_real_1.png

Figura 6: Producciones numeradas de la gramática

vscode_ll1_real_2.png

Figura 7: Conjuntos FIRST, FOLLOW y tabla LL(1) (59 entradas, 0 conflictos)

vscode_error_lexico_real.png

Figura 8: Detección de un error léxico en el terminal integrado de VS Code